


**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY**

Department of Computer Science and Engineering

# Project Proposal

## Offline Hand Sign to Text Recognition System for American Sign Language

**Course Name & Code:** UCS503 – Software Engineering

**Degree & Year:** B.Tech (3rd Year)

**Institute:** Thapar Institute of Engineering and Technology

**Submitted By:**

| | |
|---|---|
| Abhishek Srivastava | - 1024030236 |
| Keshav Aggarwal | - 1024030234 |
| Harshine Singh | - 1024030232 |
| Bhuvaneesh Singh Grewal | - 1024030235 |

**Academic Year:** 2026–27 ODD SEMESTER

## 1 Higher-order goal

This project aims to make basic sign-language communication accessible without depending on paid cloud services, third-party APIs, or pretrained models. The broader motivation is assistive and inclusive technology for the hearing- and speech-impaired community, but the The immediate engineering objective is narrower and fully within the course scope: build an end-to-end pipeline that recognizes static American Sign Language (ASL) hand signs, the 26 alphabet letters and 10 digits, from a live webcam feed, and converts them to on-screen text, using only models trained from scratch on our own machines.

## 2 Time-to-value

- The pipeline is built in independently testable stages: raw-image pre-processing, model training, and live inference. Each stage produces a verifiable artifact before the next one begins.
- We prioritize fast feedback over perfection. The WebSocket link between the browser and backend was proven first with a trivial echo handler, before any machine-learning code was written, so integration bugs surface early rather than at the end.
- Success in the first iteration means a single captured frame correctly classified end-to-end. From there, we grow the class set and tighten accuracy.

## 3 Problem Statement

Hand-sign-to-text demos that students typically build tend to fall into two categories, and Both are weak choices for a course project meant to show original engineering work.

API/cloud-dependent solutions call a hosted vision service for hand tracking or classification. These need API keys, cost money or hit rate limits, send video data off-device, and stop working offline.

Pretrained-model wrappers load an existing classifier and build a UI around it. That shows integration skill, but not the ability to design, train, and evaluate a model from scratch.

We want every learned component, the classifier that maps an image to a letter or digit,

to be trained by us, on our own data split, with no network calls at inference time. That means figuring out how to structure a from-scratch CNN training pipeline, and how to split the work across a three-person team so major components can be built in parallel, and how to serve a locally trained model to a live browser feed at acceptable latency.

# 4 Proposed Solution

## 4.1 Overview

The system is a browser-to-server "Sign-to-Text" pipeline with three decoupled parts, each owned mainly by one team member:

- Backend and CNN: a Java / Spring Boot service that exposes the frame-classification endpoint, holds the CNN training pipeline built with DeepLearning4j (DL4J), loads the trained model, and returns predictions.
- Frontend and Camera: a browser interface that accesses the webcam, captures frames, streams them to the backend, and shows the recognized letter or digit along with the accumulating text.
- Pre-processing: a Python pipeline that reads the raw ASL image dataset, cleans, and resizes it, and builds a reproducible train/test split for the CNN.

Testing runs across all three components rather than being confined to a separate module.

## 4.2 Core workflow

1. Pre-processing (offline, one-time): raw ASL images are read from the dataset's raw folder, not the provided "processed" folder, resized to $128 \times 128$, shuffled per class, and split 80/20 into our own train/test sets with a fixed random seed.
2. Training (offline, one-time per iteration): a CNN is built and trained from scratch, no pretrained weights, on the processed training set, evaluated on the held-out test set After every epoch, keep the best checkpoint.
3. Inference (live): the browser captures a frame and sends it to the backend, which runs It goes through the trained CNN and returns the predicted class.
4. Display: the frontend shows the predicted letter or digit and appends it to a running text buffer, the user watches as it builds up in real time.

### 4.3 Operational constraints for an educational, from-scratch setting

- No API keys, no third-party inference calls, no internet dependency at inference time. The trained model and server both run entirely on local hardware.
- No pretrained weights or transfer learning. The CNN's parameters are Xavier-initialized Moreover, learned entirely from our own training run.
- The dataset stays out of version control since it does not fit GitHub's practical size limits. Code lives on GitHub, the dataset separately, with its source and folder structure documented so every teammate can reproduce it locally.

## 5 Solution Approach

This is an engineering course project, so the focus is on a correct, reproducible, from-scratch training-and-serving pipeline rather than inventing new model architectures.

### 5.1 Dataset

We use the ASL-HG (American Sign Language Hand Gesture) dataset: 36,000 images across 36 classes (A-Z and 0-9), collected from 10 volunteers, released with a raw-image folder and a separate pre-processed folder. We pre-process the raw images ourselves, resizing, shuffling, and splitting from scratch, instead of using the dataset's provided split, so every step from raw pixels to the trained model is our own code.

### 5.2 Model and training: from-scratch CNN in Java

The classifier is a CNN with three convolution blocks (channels going $32 \rightarrow 64 \rightarrow 128$), each followed by batch normalization, a non-linear activation, and max-pooling to halve the spatial resolution, feeding into a small dense classification head over the 36 classes. Instead of training in Python, we build and train the model directly in Java using DeepLearning4j (DL4J), with ND4J handling the underlying tensor operations and DataVec handling image loading and label inference from the folder structure. Training uses the Adam optimizer and is evaluated after every epoch on the held-out test set, so the pipeline reports a genuine train/test accuracy split instead of a single end-of-run number.

### 5.3 Backend

The Spring Boot backend exposes a lightweight endpoint that takes a captured frame from the browser and returns the predicted label using the trained DL4J model loaded at startup. Keeping the ML runtime and the serving layer in the same language avoids the need for a model export step. (to ONNX, for example) that would otherwise be needed to run a Python-trained model outside Python.

### 5.4 Frontend and Camera

The frontend is a browser interface using the browser's built-in `getUserMedia` camera access to capture webcam frames. Captured frames go to the Spring Boot backend for classification, and the predicted letter or digit is displayed to the user while recognized characters accumulate into a running text buffer.

The frontend stays independent of the CNN implementation, talking to the backend through a defined interface so the frontend and model components can be built and tested separately.

### 5.5 CI and fast delivery enabler

Each of the three components is independently buildable and testable, so a change to Pre-processing does not require rebuilding the frontend, and changes to the CNN or backend don't require touching the camera interface.

Python pre-processing checks, Java unit tests for model loading and prediction, and Frontend integration tests catch pipeline breakage early, rather than at demonstration time.

## 6 Evaluation Criterion

Success is defined by measurable metrics tied to the core classification task.

### 6.1 Primary evaluation metric

Test-set classification accuracy: the percentage of held-out test images that the trained CNN labels correctly.

- Target: $\geq 90\%$ top-1 accuracy on our own 20% held-out split across all 36 classes.
- Measured directly from the DL4J evaluation pass over the test iterator after training completes, logged per epoch to track convergence.

### 6.2 Secondary evaluation metrics

- Per-class precision/recall, to spot which visually similar signs (letters with near-identical hand shapes) The model confuses most.
- End-to-end inference latency, the time from frame capture in the browser to a displayed prediction, to confirm the system feels usable at a live, interactive pace.
- Reproducibility: re-running the pre-processing script with the fixed seed should produce an identical train/test split, checked by the test suite.

### 6.3 Pilot validation plan

Team members will each perform all 36 signs in front of the webcam under normal indoor conditions lighting and record predicted-vs-actual labels, as a qualitative sanity check beyond the held-out test accuracy.

We will also compare accuracy and confusion patterns between our from-scratch train/test split and a quick baseline run on the dataset's own provided split, to confirm our pre-processing It is not a quiet degradation of quality.

## 7 Scalability

1. Theoretical: the system is set up to grow from static alphabet/digit signs to dynamic, full-word gestures. Pre-processing, training, and serving stay decoupled, so a future A sequence-based classifier could replace the single-frame CNN without touching the frontend capture logic, and the backend endpoint is stateless per request, so it could later accept a A short buffered sequence of frames instead of one, for dynamic gesture recognition.
2. Practical: the system runs entirely on commodity hardware. A laptop CPU is enough for Both training and inference at the current dataset scale, with no managed cloud service needed, which keeps it reproducible for every teammate and for grading.

## 8 Engine availability heuristic

The tooling needed already exists and is freely available, so there is no need to build infrastructure from scratch:

- Spring Boot for the backend web/REST layer and serving the trained CNN.
- DeepLearning4j, ND4J, and DataVec for CNN construction, training, and image data loading in Java.
- OpenCV and Pillow (Python) for raw-image reading and resizing during pre-processing.
- The browser's `MediaDevices/getUserMedia` APIs for webcam access, no An external SDK is required.
- Git/GitHub for source control, with the dataset hosted separately rather than committed to the repository.

Using these mature, freely available components lets us focus our effort on the from-scratch model and pipeline design rather than rebuilding basic infrastructure that already exists.

## 9 Project Scope and Deliverables

### 9.1 Initial Deliverable, First Iteration

- Pre-processing: raw images resized, shuffled, and split 80/20 into train/test folders.
- CNN and Backend: CNN architecture and training loop in Java/DL4J, trained from scratch on the 36-class set, plus a Spring Boot endpoint that loads the trained model and classifies a single submitted frame.
- Frontend and Camera: a minimal browser frontend for camera access, frame capture, and display of the predicted letter/digit.
- Evaluation: per-epoch test-set evaluation, with the best checkpoint saved for serving.

### 9.2 Subsequent Deliverables

- A running text buffer in the frontend that accumulates recognized letters/digits into words.
- Confusion-matrix analysis and targeted data augmentation for the most-confused sign pairs.
- An automated test suite covering pre-processing determinism, model loading, backend prediction, and frontend-backend integration.
- Documentation explaining the Spring Boot wiring, DL4J training pipeline, pre-processing workflow, and frontend-camera integration.
- Stretch goal: extend from static single-frame classification to short buffered sequences, to support a small vocabulary of dynamic, full-word signs.

## 10 Risks and Mitigations

Training from scratch on 36 classes without pretrained weights may need more epochs or data than a transfer-learning approach would. We are mitigating this by starting with a small, well-understood architecture (3 conv blocks) and only adding capacity if we see underfitting.

The pipeline crosses languages, Python for pre-processing and Java for training, and a mismatch in how images are resized, normalized, or labeled between the two could silently corrupt training data. Tests check the output folder structure and image counts per class before training is allowed to start.

Not every teammate starts with Java-web or DL4J experience. We are handling this with plain-language documentation of new concepts as they come up, so owning a module doesn't require re-deriving unfamiliar behavior in the framework from scratch.

The 36,000-image dataset is too large for practical Git version control, so it stays out of the repository, with its public source and expected local folder layout documented in the README.

## 11 Summary

This project is a fully self-trained, offline hand-sign-to-text system for the ASL alphabet Furthermore, digits are built without pretrained models or third-party API dependencies. It hits the course criteria through fast, independently verifiable pipeline stages, a clean three-way split of work across Backend and CNN, Frontend and Camera, and Pre-processing, and a measurable evaluation criterion centered on held-out classification accuracy.

It stays focused on disciplined, from-scratch engineering, our own data split, our own CNN, our own training loop in Java, while leaving a clear path toward dynamic, full-word sign recognition as future work.